Software Design Document

2/6/2026

REACH

Sponsored by Dr. Zachery Lerner, Dr. Carlo Da Cunha

Mentor: Bailey Hall

REACH Members: Taylor Davis, Lucas Larson, Clayton Ramsey, Victor Rodriguez

Version 1.1

Table of Contents

Introduction …….. 3
Implementation Overview ……...……. 3
Architectural Overview ……..…….. 4
Architectural Diagram Overview ……. 4
System Components and Responsibilities ……..…….. 4
Communication and Data Flow …… 7
External Dependencies ……...……..…… 7
Architectural Decisions and Rationale …….. 7
Component-Level Design ……...…….. 8
Gesture Data Collection ……...……. 8
Gesture Model Training …….……..…. 9
Gesture Inference Engine ……..……..….. 9
Real-Time Gesture Recognition ……..……..…… 9
MuJoCo Robot Simulation Environment …… 10
Gesture-Controlled Robot Controller ……..…… 11
Demo / Integration Runner …….……. 11
Cross-Cutting Concerns ……..……. 12
Implementation Plan ……...……. 13
Development Timeline Diagram ……...…….……. 13
Major Development Stages …….. 13
Technical Risks and Trade-Offs …….. 14
Conclusion ……..……. 15

1. Introduction

In recent years, advances in robotics and artificial intelligence have opened new possibilities for assistive technologies aimed at improving independence and quality of life for individuals with motor impairments. Robotic assistive devices, particularly wearable and semi-autonomous robotic arms, have shown significant potential in helping users perform everyday tasks such as reaching, grasping, and object manipulation. However, many existing systems rely on rigid, preprogrammed behaviors that struggle to adapt to the diverse needs, environments, and movement patterns of individual users. This capstone project, REACH (Reinforcement Learning for an Assistive Robotic Arm), addresses these challenges by leveraging reinforcement learning to create a more adaptive and intelligent control system for assistive robotics.

The primary objective of the REACH project is to develop a reinforcement learning–based control framework that enables robotic limbs to perform functional, human-centered tasks in a safe and efficient manner. Using a physics-based simulation environment built with MuJoCo, the system allows the robotic arm to learn complex behaviors such as reaching for objects, grasping, and interacting with its environment through trial and error. By training the arm in simulation, the project reduces risk to users and hardware while enabling rapid experimentation with control strategies, reward functions, and task definitions.

This project was undertaken in partnership with NAU Biomechatronics Lab led by Dr Zach Lerner as well as Dr. Carlo da Cunha from the NAU AI robotics lab. They are a research partner dedicated to developing and evaluating assistive devices, with the goal of moving flexible, adaptive robotic systems from the lab toward real-world use. They approached our team because they needed a control system that could adapt to different users and environments instead of relying on fixed, preprogrammed behaviors.

Through the integration of modern reinforcement learning algorithms, sensor feedback, and structured task environments, REACH aims to produce motion policies that generalize across different object positions and task scenarios. The system is designed with extensibility in mind, allowing future integration with real-world hardware and robotic middleware such as ROS for sim-to-real transfer. Performance metrics such as task completion rate, efficiency, and stability are used to evaluate the learned behaviors and guide iterative improvements.

By focusing on adaptability and user-centered design, the REACH project contributes to ongoing efforts in assistive robotics research by demonstrating how learning-based approaches can improve autonomy and flexibility in robotic assistance. Ultimately, this project seeks to support the development of intelligent assistive systems that better respond to real-world variability, helping bridge the gap between laboratory robotics research and practical, life-enhancing technologies for individuals with physical disabilities.

FOR TJ

The assistive robotics industry focuses on developing technologies to enhance independence and quality of life for individuals with physical limitations. The global assistive technology market is valued at over $28 billion as of 2025, with robotics growing as a segment. A major area is upper-limb rehabilitation for stroke survivors, who often have persistent motor deficits affecting daily activities like eating, grooming, and reaching. Current solutions include prosthetics and exoskeletons, but they often rely on rigid, preprogrammed movements that struggle in dynamic real-world settings.

At Northern Arizona University, Dr. Zach Lerner of Mechanical Engineering and Prof. Carlo R. da Cunha of Electrical Engineering lead research in biomechanics and assistive technologies. Their work bridges mechanical design, control systems, and human-machine interaction, positioning NAU as a hub for rehabilitation robotics. Dr. Lerner's biomechanics expertise informs the mechanical design and human factors, while Prof. da Cunha's control systems work addresses how robotic systems interpret and respond to human intent. Their collaboration aims to address the gap between rigid robotic control and natural human movement.

2. Implementation Overview

The solution developed for the Biomechatronics Lab focuses on creating a flexible and efficient reinforcement-learning (RL) framework for training and operating a robotic arm mounted at a user's elbow. The primary objective of this system is to enable iterative learning of arm movements through simulation, allowing trained policies to be evaluated and refined before deployment. Design decisions were shaped by constraints such as compatibility with existing

research tools, the need for rapid iteration, and the computational demands of reinforcement-learning workflows.

At a high level, the framework is built around a Python-based reinforcement-learning pipeline that integrates simulation, training, and visualization into a cohesive system. Python was selected as the core programming language due to its strong ecosystem for scientific computing and machine learning, as well as its native compatibility with simulation tools such as MuJoCo. Libraries including NumPy, Stable Baselines, SciPy, and Gymnasium provide the numerical, learning, and environment abstractions required to support policy training at scale.

MuJoCo serves as the system's primary visualization and physics simulation engine. It enables real-time visualization of the robotic arm and supports XML-based model definitions, allowing trained policies to be evaluated within a physically accurate simulated environment. Within the framework, MuJoCo's role is to display training progress and final policy behavior, providing visual feedback on the effectiveness of reinforcement-learning strategies.

The training architecture itself is centered on an iterative reinforcement-learning loop. Policies are trained over predefined time horizons and repeatedly refined until their performance falls within acceptable parameters. To support this process, Stable Baselines (backed by PyTorch) was integrated to balance performance, modularity, and ease of implementation. This approach allows for rapid experimentation with learning algorithms while maintaining extensibility for future enhancements.

Overall, the framework is designed to be modular, efficient, and adaptable, enabling reliable training of robotic arm movements while remaining easy to modify and deploy in research environments. Alternative technologies such as TensorFlow-based frameworks, alternative simulators like RaiSim, or lower-level languages such as C++ were evaluated but ultimately deemed less suitable due to reduced compatibility, increased complexity, or diminished development efficiency for the project's goals

3. Architectural Overview

This section describes the high-level architecture of the REACH system and explains how its major components interact to support reinforcement learning based control of an assistive robotic arm. The architecture is designed to support simulation-first development, modular experimentation, and a clear pathway toward ROS-based sim-to-real deployment.

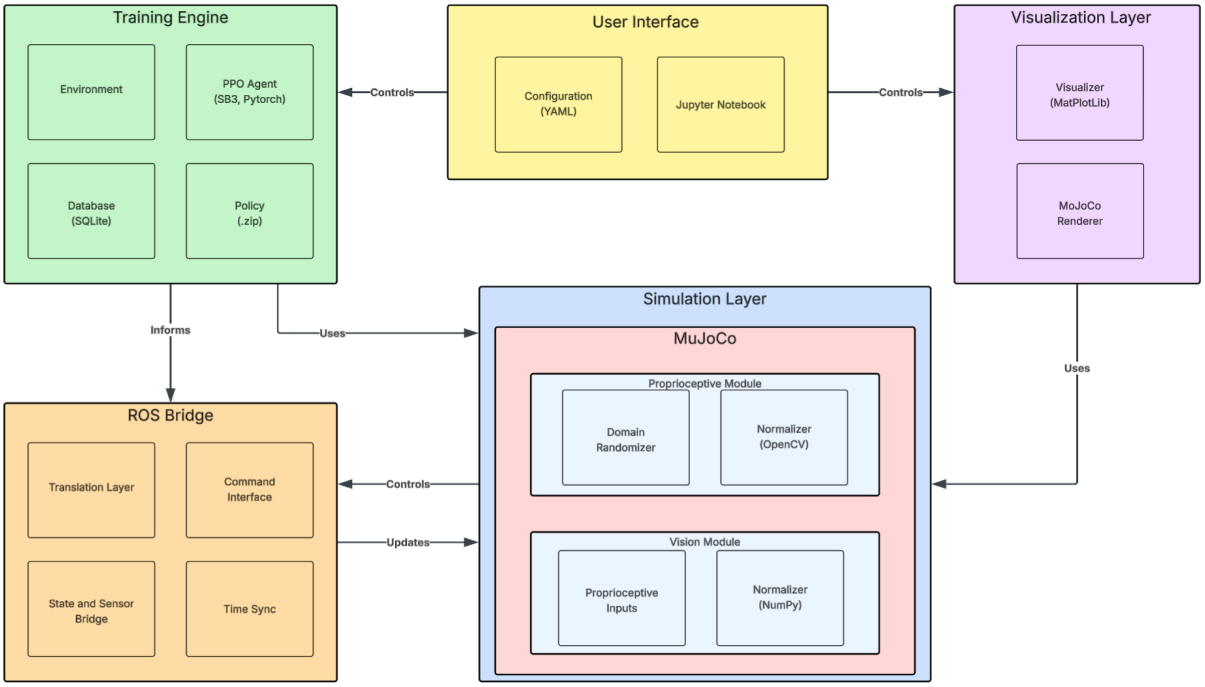

Figure 1

3.1 Architectural Diagram Overview

Figure 1 illustrates the REACH system architecture. The diagram emphasizes separation of concerns between learning, simulation, communication, and user interaction. The Training Engine drives the reinforcement learning loop, the Simulation Layer provides physics-based state evolution via MuJoCo, and the ROS Bridge serves as a translation and communication boundary between the learning system and ROS-compatible control and sensing interfaces.

The User Interface provides configuration and experiment control, while the Visualization Layer enables real-time and offline inspection of simulation and training behavior. Data and control flow through the system using a combination of synchronous method calls (within the training loop) and asynchronous messaging (via ROS-style interfaces).

3.2 System Components and Responsibilities

3.2.1 Training Engine

The Training Engine is responsible for policy learning, experiment execution, and data persistence. It contains the reinforcement learning agent (implemented using Stable-Baselines3 and PyTorch), the environment interface, and local storage for trained policies and experiment metadata.

The PPO agent interacts synchronously with the environment to collect observations, generate actions, and receive reward signals. Training metrics and policies are stored locally using a lightweight SQLite database and serialized policy files.

Responsibilities:

- Reinforcement learning policy training and optimization
- Management of experiment episodes and rollouts
- Storage of trained policies and training metadata
- Coordination with the simulation layer for state transitions
- Informing the ROS Bridge of control and evaluation context

3.2.2 Simulation Layer (MuJoCo)

The Simulation Layer provides a high-fidelity physics-based model of the assistive robotic arm and its environment using MuJoCo. It is responsible for simulating robot dynamics, sensor outputs, and task interactions.

The simulation includes modular sensing pipelines, such as proprioceptive and vision-based modules, along with normalization and domain randomization components. These modules enable realistic observation generation and support robustness for future sim-to-real transfer.

Responsibilities:

- Physics simulation of the robotic arm and environment
- Generation of proprioceptive and visual observations
- Application of domain randomization during training
- Execution of control commands from the Training Engine or ROS Bridge

3.2.3 ROS Bridge

The ROS Bridge acts as a translation and communication layer between the reinforcement learning system and ROS-compatible control and sensing interfaces. It decouples the training logic from ROS-specific messaging patterns and enables future deployment to physical hardware without modifying the learning pipeline.

The bridge converts policy outputs into robot control commands and publishes state and sensor updates using ROS-style interfaces. It also handles time synchronization to ensure consistency between simulation time and ROS time.

Responsibilities:

- Translation between RL actions and ROS control commands
- Publication of robot state and sensor data
- Time synchronization and control-rate management
- Acting as a boundary for sim-to-real transfer

3.2.4 User Interface

The User Interface provides experiment configuration and high-level control of the training process. Configuration files (YAML) define environment parameters, reward structures, and training hyperparameters, while Jupyter Notebooks enable interactive experiment execution and analysis.

This component does not participate directly in the training loop but instead controls and informs the Training Engine and Visualization Layer.

Responsibilities:

- Experiment configuration and parameter management
- Interactive execution and monitoring of experiments
- High-level control of training and evaluation workflows

3.2.5 Visualization Layer

The Visualization Layer supports both real-time and offline analysis of system behavior. It includes Matplotlib-based plotting for training metrics and a MuJoCo renderer for visual inspection of the simulated environment.

Visualization operates independently of the core training loop to avoid performance interference and supports both debugging and presentation needs.

Responsibilities:

- Visualization of training metrics and learning progress
- Rendering of simulation state and robot motion
- Supporting qualitative evaluation of learned policies

3.3 Communication and Data Flow

The system uses a combination of synchronous and asynchronous communication mechanisms:

- **Synchronous communication** occurs between the Training Engine and Simulation Layer during reinforcement learning, where each action immediately produces a new observation and reward.
- **Asynchronous communication** occurs through the ROS Bridge, where state updates and control commands follow publish/subscribe patterns consistent with ROS-based systems.

This hybrid approach enables high-throughput training while maintaining compatibility with real-world robotic control architectures.

3.4 External Dependencies

The REACH system relies on several external frameworks and libraries:

- **MuJoCo** for physics-based simulation
- **Stable-Baselines3 and PyTorch** for reinforcement learning
- **ROS (via ROS Bridge)** for robotic communication and deployment compatibility
- **NumPy and OpenCV** for data normalization and processing
- **Matplotlib and Jupyter** for visualization and experimentation

These technologies were selected for their maturity, performance, and widespread adoption in robotics research.

3.5 Architectural Decisions and Rationale

Several non-trivial architectural decisions were made in the design of REACH:

- **Simulation-first architecture:** Training is performed entirely in simulation to reduce risk, enable rapid iteration, and avoid hardware damage.

- **Explicit ROS boundary:** ROS integration is isolated to the ROS Bridge to prevent ROS-specific complexity from propagating into the learning system.
- **Modular sensing pipelines:** Separate proprioceptive and vision modules support extensibility and future sensor integration.
- **Configuration-driven experiments:** YAML-based configuration improves reproducibility and simplifies experimental variation.
- **Hybrid sync/async communication:** Synchronous learning loops maximize training efficiency, while asynchronous ROS messaging enables real-world compatibility.

4. Component-Level Design

This section describes the internal structure of each major component in the REACH system, its responsibilities, and its public interfaces. Components are organized to align with the system architecture and avoid overlapping ownership.

4.1 Gesture Data Collection

Captures labeled hand/arm motion sequences from a camera feed using MediaPipe and stores them as JSON datasets for training. This component owns data acquisition and labeling.

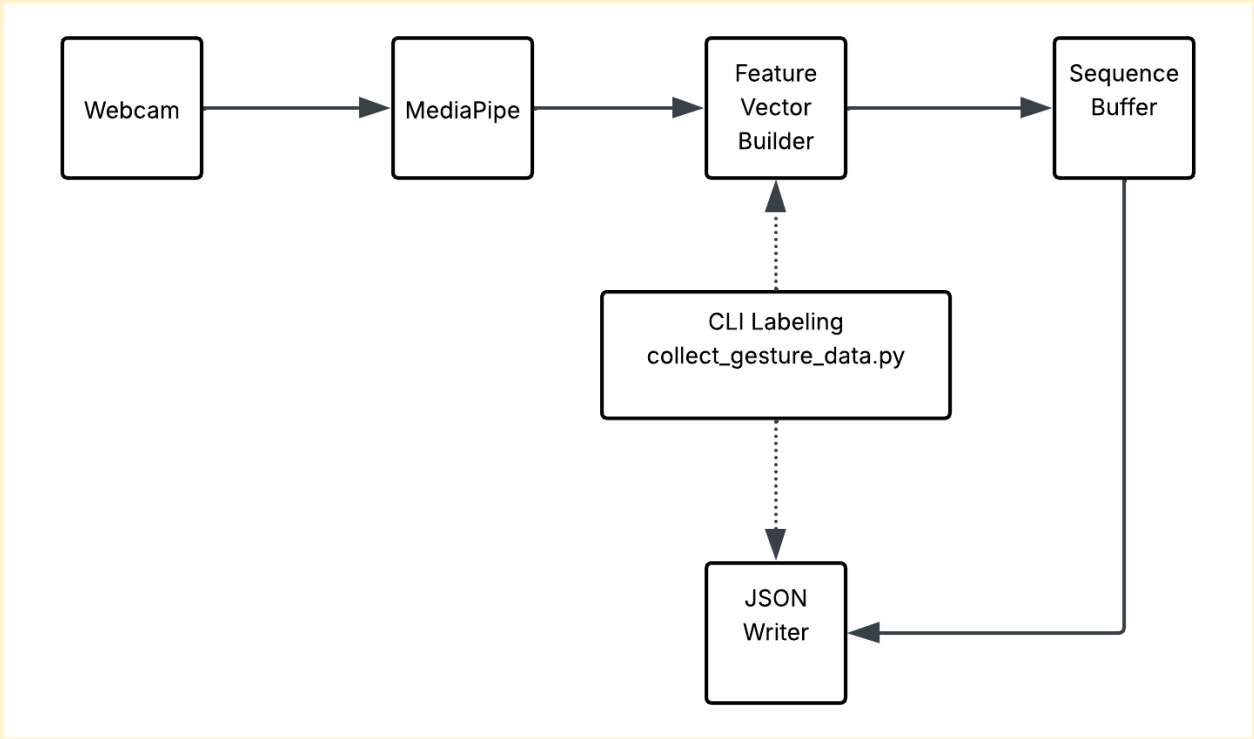

Figure 2

Public Interface

- Inputs: live webcam frames
- Outputs: JSON sequences in scenes/image_recognition/data/<gesture>/
- Execution: collect_gesture_data.py (CLI)
- Key responsibilities:

- Collects sequences for each gesture class
- Ensures consistent feature vector format (144 features/frame)

4.2 Gesture Model Training

Trains the gesture classifier using collected keypoint sequences. Produces a model checkpoint used in inference and integration.

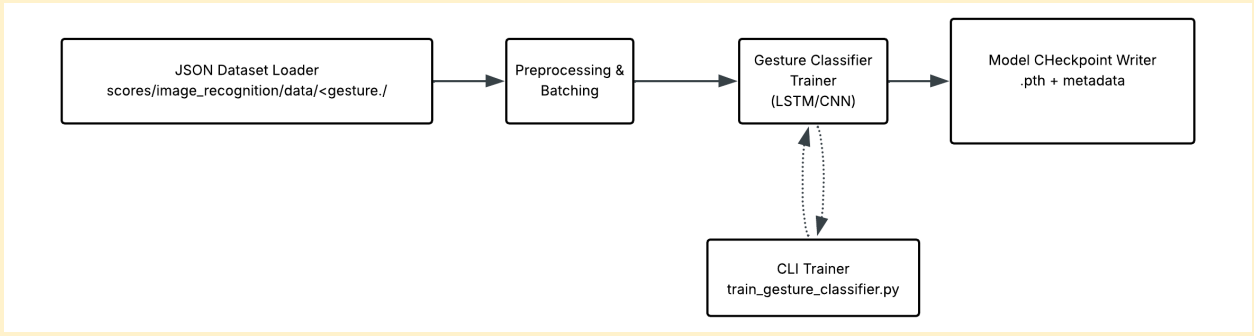

Figure 3

Public Interface

- Inputs: JSON sequences from data collection
- Outputs: .pth model checkpoint
- Execution: train_gesture_classifier.py (CLI)
- Key responsibilities:
  - Train LSTM/CNN classifier
  - Save model + metadata for inference

4.3 Gesture Inference Engine

Runs inference on keypoint sequences and returns gesture probabilities. This is the reusable "engine" used by both real-time inference and robot control.

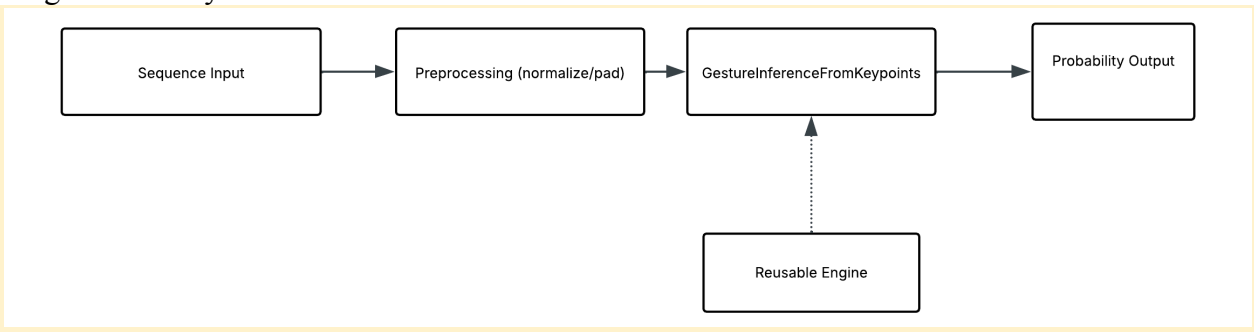

Figure 4

Public Interface

- Inputs: sequence of 144-feature vectors
- Outputs: probability dictionary, predicted class
- API: GestureInferenceFromKeypoints class
- Key responsibilities:
  - Normalize/pad sequences
  - Return gesture probabilities with confidence

4.4 Real-Time Gesture Recognition

Performs live gesture recognition and logs results to CSV. Used for testing and validating accuracy without running the robot.

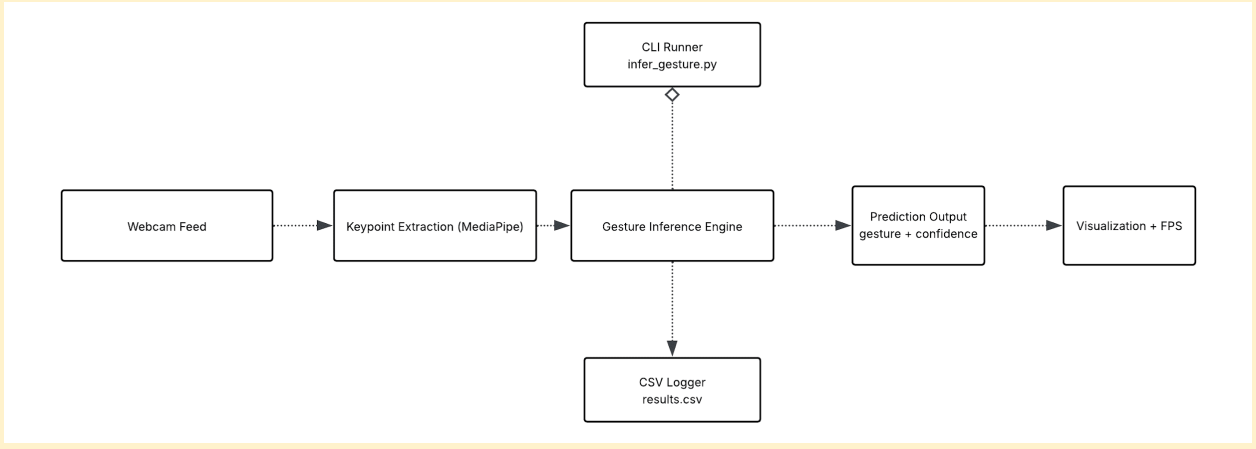

Figure 5

Public Interface

- Inputs: webcam feed
- Outputs: predicted gesture, confidence, CSV logs
- Execution: infer_gesture.py (CLI)
- Key responsibilities:
    - Real-time visualization
    - Performance tracking (FPS)
    - Event logging

4.5 MuJoCo Robot Simulation Environment

Simulated robot environment that exposes a step-based API and state observations. Owns physics state, reward shaping, and episode termination.

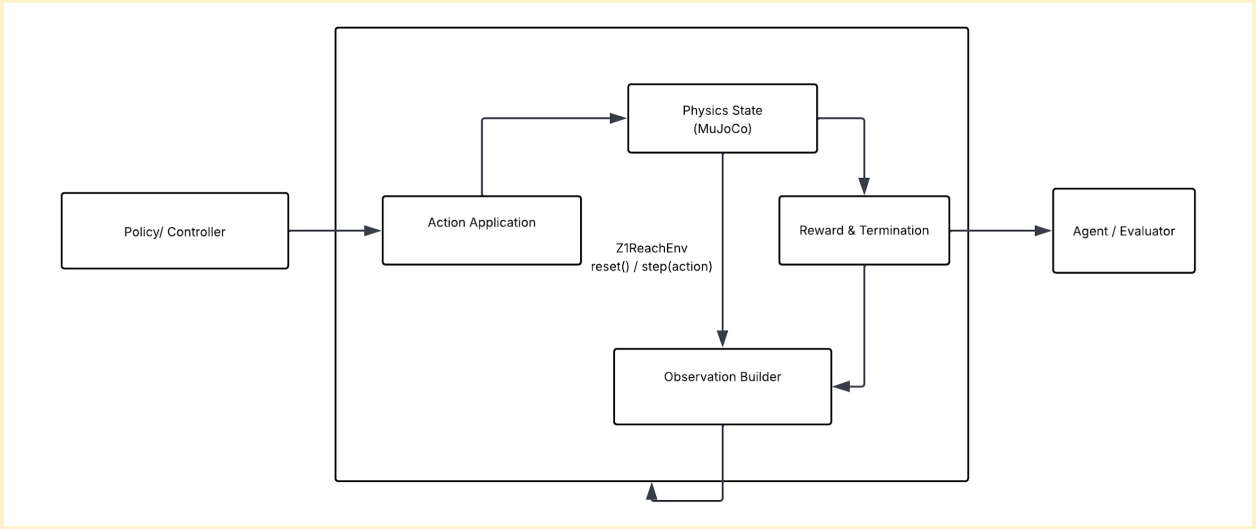

Figure 6

Public Interface

- Inputs: action vectors (joint control)

- Outputs: observation, reward, termination flags
- API: Z1ReachEnv.reset() / Z1ReachEnv.step(action)
- Key responsibilities:
    - Apply control actions
    - Compute observation and reward
    - Handle resets and termination

4.6 Gesture-Controlled Robot Controller

Integrates the gesture pipeline with the robot environment. Maps gesture classes to robot actions and executes them in the simulation loop.

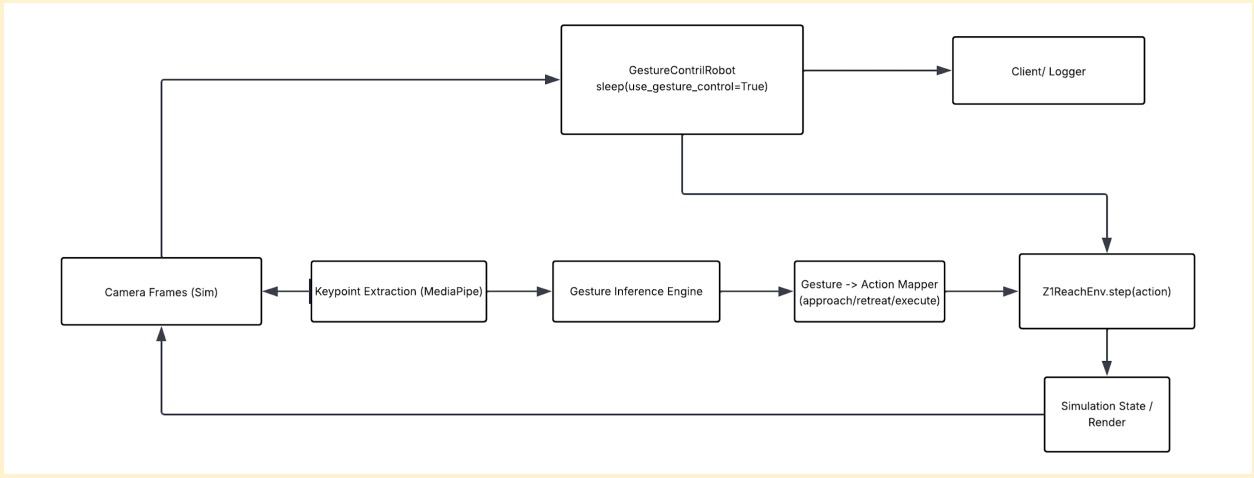

Figure 7

Public Interface

- Inputs: camera frames or gesture sequence
- Outputs: robot actions + gesture info
- API: GestureControlledRobot.step(use_gesture_control=True)
- Key responsibilities:
    - Render camera frames from simulation
    - Extract keypoints, run inference
    - Map gesture → action (approach/retreat/execute)
    - Step simulation with resulting action

4.7 Demo / Integration Runner

High-level script that demonstrates the end-to-end gesture-to-robot pipeline, with optional webcam control and visualization.

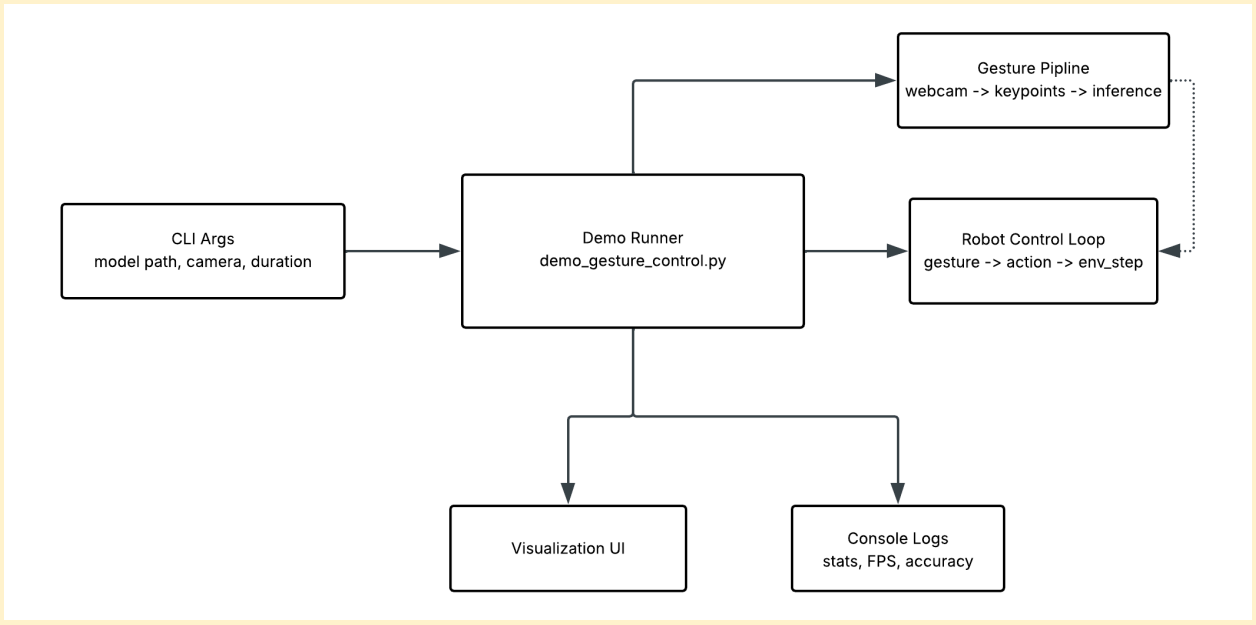

Figure 8

Public Interface

- Inputs: CLI args (model path, camera name, duration)
- Outputs: on-screen visualization + console logs
- Execution: demo_gesture_control.py
- Key responsibilities:
    - Orchestrate webcam + robot control
    - Present live feedback and statistics

4.8 Cross-Cutting Concerns

Cross-Cutting Concerns (applies to all components)

- Error Handling
    - Graceful handling when model files or cameras are missing
- Performance
    - Real-time inference target ≥15 FPS
    - Gesture confidence thresholds to reduce false positives
- Logging and Observability
    - Gesture events logged to CSV
    - Episode metrics optionally logged for simulation
- Extensibility
    - New gesture class can be added by expanding:
        - Data collection classes
        - Model output classes
        - Gesture-to-action mapping

5. Implementation Plan

5.1 Development Timeline Diagram

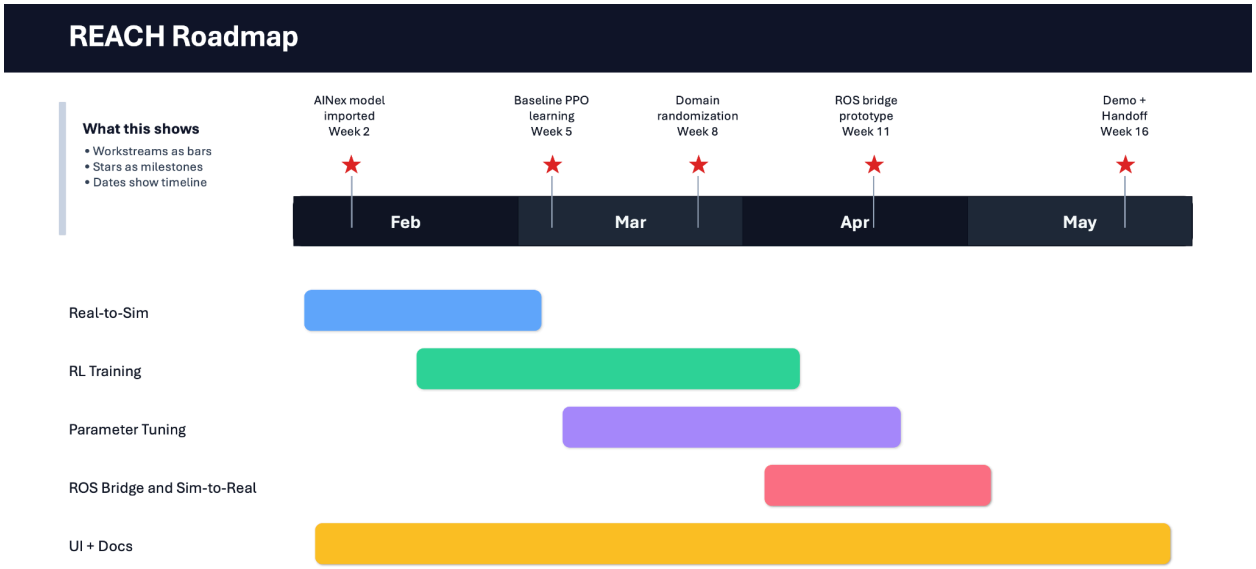

Figure 8

5.2 Major Development Stages

Real-to-Sim Preparation
Initial development focuses on preparing the AINex robot platform for simulation-based reinforcement learning. This includes extracting URDF models and motion presets from the physical robot, importing them into MuJoCo, and tuning physics parameters to ensure stable and realistic motion. This stage establishes the simulation environment needed for policy development while the primary assistive arm hardware remains unavailable.

Reinforcement Learning Pipeline Development
Once a stable simulation environment is established, development shifts to the reinforcement learning pipeline. This includes environment construction, reward function design, hyperparameter tuning, and training workflow development using Stable Baselines3 and PyTorch. HPC resources are leveraged for longer training runs, allowing policies to be iteratively refined within a controlled simulation environment.

Simulation Feature Expansion
After baseline training capability is established, additional features are incorporated to improve robustness and realism. These include sensor modeling, observation normalization, and domain randomization techniques that help prepare policies for eventual real-world deployment. Visualization and monitoring tools are also expanded to support debugging and evaluation of learned behaviors.

Sim-to-Real Integration Preparation

Development then focuses on building infrastructure that will allow trained policies to transfer to physical hardware. This includes implementing the ROS Bridge for communication between simulation and robot control systems, defining standardized action and observation interfaces, and ensuring trained policies can be exported and reused outside the simulation environment.

User Interface and Tooling
Parallel to the core development work, usability features are developed to support experimentation and reproducibility. These include YAML-based configuration files, automated environment setup tools, improved documentation, and simple mechanisms for loading scenes, running experiments, and visualizing results.

System Testing and Validation
Testing occurs continuously throughout development, with emphasis on integration once major components are functional. Policies are evaluated in simulation, gesture and perception pipelines are validated where applicable, and workflow testing ensures the framework can be reliably used by researchers and future project teams.

Documentation and Final Delivery
The final phase focuses on consolidating documentation, stabilizing the framework, and preparing materials for project handoff. This includes developer documentation, experiment guides, and architectural explanations that support future continuation of the work, including eventual deployment on the assistive robotic arm.

5.3 Technical Risks and Trade-offs

As with many robotics and reinforcement learning projects, several technical risks and trade-offs may influence development progress and system performance. One primary risk involves the challenges associated with simulation-to-real transfer. Even with domain randomization and careful modeling, simulated physics may not fully capture real-world dynamics, which can lead to instability or reduced performance when policies are deployed on physical hardware.

Integration with ROS introduces additional complexity. While ROS provides valuable interoperability with robotic platforms, it also adds communication overhead, version compatibility considerations, and configuration challenges that can impact development speed and system reliability.

Another trade-off relates to gesture recognition capabilities. Gesture-based control provides an intuitive interaction modality, but training robust models can be limited by dataset size, environmental variability, and camera quality. These factors may affect recognition accuracy and responsiveness in real-world scenarios.

Finally, hardware availability remains an external dependency. Delays in the assistive arm development have required a temporary pivot to the AINex robotic platform to maintain progress. While this enables continued experimentation, differences between platforms may require additional adaptation effort during future sim-to-real deployment.

These risks are being mitigated through modular architecture, simulation-first development practices, and ongoing iterative testing to maintain flexibility as project conditions evolve.

6. Conclusion

This design document outlines the architectural foundation and implementation strategy for the REACH system, a reinforcement learning–driven framework intended to support adaptive control of assistive robotic arms. By emphasizing a simulation-first development approach, modular system architecture, and clear separation between learning, simulation, communication, and user interaction layers, the design supports both near-term experimentation and long-term real-world deployment.

The proposed architecture enables safe, efficient policy development in simulation while maintaining compatibility with physical robotic platforms through the ROS Bridge interface. The inclusion of configurable training pipelines, visualization tools, and structured user workflows ensures that the system remains accessible to researchers and extensible for future development. Additionally, the integration of gesture recognition components demonstrates how multimodal interaction can enhance assistive robotic control.

Overall, this design provides a practical blueprint for building an adaptable reinforcement learning framework capable of evolving alongside both hardware availability and research needs. As development progresses, this document will continue to serve as a living reference, guiding implementation decisions and ensuring alignment with project objectives. Ultimately, the REACH project aims to contribute toward more flexible, intelligent assistive robotic technologies that can better respond to the diverse needs of users in real-world environments.